SABZ

AI-ADAPTIVE TRAFFIC SIGNALS

Judge Q&A Preparation Guide

Garrison 4th International AI Championship - AI in Urban Tech

Team: Abdur Rehman, M. Bilal, Awais, Dawood

Lahore Garrison University - B.S. Computer Science

SECTION 1: WHAT EXACTLY IS SABZ

SABZ (Urdu for green) is an AI system that controls traffic signals. Instead of running on a fixed timer that gives every road the same green no matter what, SABZ watches real traffic through cameras, counts the cars waiting on each road, and decides second by second which road deserves the green light next.

The decision-maker is a small neural network called a Deep Q-Network (DQN), 5,506 parameters, trained from scratch by the team in a traffic simulator. It learns one job: keep the total waiting time of all cars as low as possible. It runs on an ordinary CPU and makes each decision in about 0.02 milliseconds, so it can easily keep up with real traffic.

One thing is not learned: safety. A hard rule sits outside the neural network and can never be changed by training: the signal is never allowed to cut a green light while any car is within 70 metres of the stop line. Even if the AI asks for a switch, the rule overrides it. Safety is guaranteed by design, not by hope.

Status, stated honestly: SABZ is a working trained system in simulation, benchmarked against the industry-standard actuated controller. It is not yet deployed on a real road. The camera pipeline has been proven on real recorded footage with YOLOv8 vehicle detection, but the live closed loop (camera feeding the brain feeding a real signal) is the next milestone, not a finished claim.

In one line for a non-technical judge: SABZ is a traffic signal that can see. It gives green to the road that actually needs it, learns to get better at it, and can never make an unsafe switch.

SECTION 2: THE FIVE QUESTIONS EVERY JUDGE ASKS

WHY

Q: Why does this problem matter?

A: Lahore's registered vehicles went from about 378,000 in 2014 to 4.1 million in 2024, an elevenfold rise in a decade (Excise and Taxation Department, Punjab). The average Lahore commute is now around 34.8 minutes (Numbeo 2026 Traffic Index, via Lahore Traffic Police). Every one of those minutes at a red light burns fuel, pumps CO_2, and steals time from millions of people. One published study measured the individual cost

of congestion for Lahore drivers at up to PKR 1,136.88 per day (Nawaz et al., SAGE Open, 2024). This is not an abstract problem; it is a daily tax on the whole city.

Q: Why AI, and not just better timers?

A: A timer cannot see. A fixed 25-second green gives an empty road the same time as a road with 20 waiting cars, and an actuated controller only reacts with simple if-then rules (if my queue is empty and someone waits, switch). Neither of them learns, neither anticipates, and neither adapts to the rhythm of a specific intersection: the school rush at 7:30, the Friday mosque surge, the night when almost nobody is out. A learning controller watches patterns over thousands of episodes and builds a policy for each situation. Our benchmark shows the learned policy already beats or matches the best hand-tuned actuated logic in 2 of 3 traffic scenarios.

Q: Why now?

A: Three things converged: cameras are cheap enough to mount on every signal pole, a trained neural network this small (5,506 parameters) runs on commodity hardware with no GPU, and the team had access to real measured traffic data (NGSIM) to calibrate the simulator. Five years ago the sensing was too expensive; in five years the cities that move first will own this layer. The window is open right now.

WHO

Q: Who is SABZ for?

A: First, the commuters of Lahore and other Pakistani cities who lose hours every week at signals. Second, city traffic authorities who need a system that installs on existing signal poles and existing cameras, without digging up roads for in-ground sensors. Third, eventually, any city in the developing world where traffic is mixed, chaotic, and nothing like the orderly lanes the Western systems were designed for.

Q: Who built it?

A: Four B.S. Computer Science undergraduates at Lahore Garrison University: Abdur Rehman (team lead), M. Bilal, Awais, and Dawood. The DQN was written from scratch in NumPy, not imported from a library, and trained on their own machines. No corporate lab, no grant, no borrowed model.

Q: Who benefits if it works?

A: Drivers (less waiting), the city (less fuel burned at idle, less CO_2, fewer red-light crashes from the safety guarantee), and Pakistan's tech story (a homegrown AI system for a Pakistani problem, built by Pakistani students).

WHAT

Q: What is SABZ, technically?

A: A closed loop with four parts: (1) cameras count vehicles per approach within 120 metres, (2) the counts become an 18-number state vector, (3) a Deep Q-Network picks one of two actions every second, hold the green or switch the phase, (4) the signal executes, except a hard safety rule can veto any switch that would strand a car within 70 metres of the stop line. The loop repeats every second.

Q: What does it actually do that a timer cannot?

A: It holds green longer when it sees a long queue still discharging, cuts green early when the road is empty and someone waits across, and learns location behavior: school zones get different timing than night traffic, because the state includes where the intersection is and what time it is. A timer does the same thing at 3 AM and 6 PM. SABZ does not.

Q: What makes it different from existing products?

A: SCOOT (Siemens) and SCATS (Sydney) are the incumbents. They are expensive, built for orderly Western traffic, and need in-road sensors. SABZ is camera-only (no road digging), its brain is 5,506 parameters that runs on a cheap CPU, it carries a hard dilemma-zone safety guarantee the incumbents do not advertise, and its state includes local context the incumbents ignore: mosques and Friday prayer surges, schools, hospitals, time of day. It is designed for Pakistani intersections first, not adapted to them.

WHERE / WHEN

Q: Where would this deploy?

A: The pilot vision is a single real intersection in Lahore, one signalized crossing, cameras on the existing poles, the brain running on a small computer in the signal cabinet. From there, corridor by corridor: once one intersection works, the same brain architecture trains for each new junction with its own traffic patterns.

Q: When does it work best, and when does it struggle?

A: It shines in variable demand: rush hours, school zones, asymmetric flows where one road is busy and the cross road is empty. That is exactly where fixed timers waste the most green. It is honest to say the balanced scenario, where all four roads are equally busy, is the hardest: our current brain ties the actuated controller there (11.9s vs 11.9s) instead of beating it. Training continues specifically to break that tie.

HOW

How it works, end to end, in five steps:

- 1. SEE: cameras watch each of the four approaches out to 120 metres. The system counts queued cars and approaching cars per road. (In simulation, camera counts include realistic detection noise; on real footage, YOLOv8 does the counting.)
- 2. THINK: the counts plus the current signal phase, how long green has run, where the intersection is (near a school? a mosque? is it Friday? what hour?), and how long the red road has been waiting, form an 18-number state. The DQN scores two actions, hold or switch, and picks the better one. This takes about 0.02 milliseconds.
- 3. CHECK: before any switch executes, the safety rule scans the 70-metre dilemma zone on the green approaches. If any car is inside it, the switch is vetoed and green extends. The network cannot override this; it sits outside the network.
- 4. ACT: the signal switches (3 seconds yellow, 2 seconds all-red clearance) or holds. Cars discharge, queues shrink, the clock moves one second.
- 5. LEARN: during training, every second produces a reward signal, negative total queue, so the brain feels every car-second of waiting. Over tens of thousands of simulated episodes it reshapes its policy toward lower delay. In deployment, the same loop keeps refining the policy day by day.

SECTION 3: TECHNICAL Q&A (25 QUESTIONS)

Q: What is the exact state space?

A: 18 numbers: 4 queue counts per approach divided by 20, 4 approaching-vehicle counts within camera range divided by 10, current phase (1 for NS, 0 for EW), green elapsed divided by 60, 6 location features (near_school, near_mosque, near_office, near_hospital, is_friday, hour_of_day divided by 24), and 2 starvation features (red_wait: how long the red phase has waited, divided by 300; max_q_age: the oldest queued vehicle's wait, divided by 300). The starvation features were added after the

first training run showed the brain could hold green too long and starve a phase; the old 16-dim state was blind to starvation.

Q: What is the action space?

A: Two discrete actions: 0 = hold the current green, 1 = switch the phase. The switch still passes through the safety veto and the minimum-green rule before it executes, so the action is a request, not a command.

Q: Describe the network architecture.

A: A multilayer perceptron: 18 inputs, 64 ReLU hidden units, 64 ReLU hidden units, 2 Q-value outputs. 5,506 parameters total, written from scratch in NumPy with hand-derived backprop and Adam. No PyTorch, no TensorFlow. One forward pass takes about 0.02 milliseconds on CPU, which is why real-time control is trivial for it.

Q: Why did you write your own DQN instead of using PyTorch or Stable-Baselines3?

A: Three reasons. First, control: when training plateaued, the team could change the exact update rule, the replay sampling, and the target sync schedule instead of fighting a library's assumptions. Second, deployment: a 5,506-parameter NumPy model runs anywhere, including a cheap computer in a signal cabinet, with zero dependency weight. Third, proof of understanding: at a student championship, 'we wrote the learner ourselves' answers the 'did you just pip install this' question before it is asked.

Q: How is the reward function designed?

A: Reward per sim-second is negative total queue divided by 40, minus 0.05 times the oldest queue wait (capped). Every queued vehicle-second is one second of human delay, so minimizing the reward is exactly minimizing delay; this is Little's law made into a reward. There is deliberately no penalty for switching: the 5-second yellow plus all-red transition already costs delay through the queue term, and an explicit switch penalty was measured to make the brain switch-averse and suppress needed phase changes.

Q: What is Double DQN and why do you use it?

A: Standard DQN uses one network to both pick and evaluate the best next action, which systematically overestimates Q-values; when Q(hold) and Q(switch) are both overestimated, the choice becomes a coin flip. Double DQN splits the job: the online

network picks the argmax action, the target network evaluates it. This killed the overestimation that was destabilizing early training.

Q: What is prioritized experience replay?

A: A 60,000-transition ring buffer where rare, high-error transitions (like the first seconds of a collapse) are replayed more often instead of drowning in thousands of boring uniform samples. Sampling uses proportional prioritization with alpha 0.6 and importance weights annealed toward 1.0. This is one of the fixes that ended the seed-fragile collapse of the first training run.

Q: Walk me through the training process.

A: It starts with behavioral cloning: the network imitates the hand-tuned actuated controller for a supervised warm start, so it begins as a sensible controller instead of a random one. Then DQN refinement runs in 40,000-episode blocks: each episode is 200 sim-seconds with randomized scenarios (asymmetric, balanced, school, mosque Friday surges, office tidal flows, accidents, night traffic), epsilon-greedy exploration starting at 1.0 decaying to 0.05, batch size 128, a gradient update every 2 env steps, and target network sync every 4,000 steps. Every 2,000 episodes the brain is scored on the exact benchmark; the best checkpoint is kept. Current status: about 42,000 episodes, best benchmark delay 7.3 seconds, training still running because the balanced scenario is still tied, not won.

Q: What are the benchmark numbers, exactly?

A: The benchmark is 3 scenarios times 3 random seeds times 3,600 sim-seconds each, safety on: asymmetric, actuated 5.0s average delay vs DQN 5.0s, WIN; balanced, actuated 11.9s vs DQN 11.9s, LOSE (a tie is scored as a loss); school zone, actuated 5.3s vs DQN 5.0s, WIN. The production brain's best mean delay across the full benchmark is 7.3 seconds. These are the verified numbers from the training log; the balanced tie is the current frontier.

Q: How does it beat actuated control?

A: The actuated controller is a fixed set of if-then rules: switch when my queue is empty and someone waits, respect min and max green. It reacts; it never anticipates and never adapts to location or time. The DQN learned policies the rules do not contain: holding green through a discharging platoon it can see approaching, cutting green early on an empty road at night, and shaping green around school and Friday patterns via its location features. On asymmetric and school traffic this already wins; on balanced traffic it currently ties, which is why training continues.

Q: Explain the safety hard rule in detail.

A: Every sim-second, before any switch executes, the simulator checks the dilemma zone: 0 to 70 metres from the stop line on the green approaches. If any vehicle is inside it, a switch decision is overridden to stay, and green extends up to the 60-second maximum. Switching then goes through 3 seconds of yellow and 2 seconds of all-red clearance. The physics: at 12 m/s a car needs about 24 metres to stop comfortably and the 5-second clearance covers 60 metres, so nobody ever gets a surprise yellow inside the danger zone. The rule lives in the simulator, outside the neural network; no weight update can ever change it. With safety on, executed violations are always zero by construction, and the log counts every time a controller tried an unsafe switch so we can see the rule working.

Q: How fast is inference, really?

A: Measured: about 0.02 milliseconds per decision on CPU (2,000 timed forward passes). The control loop runs once per second. The brain is roughly 50,000 times faster than it needs to be, so inference speed is a solved problem, not a risk.

Q: What prevents one road from starving while another gets endless green?

A: Two starvation features in the state (red_wait and max_q_age) let the brain see waiting time directly, the reward includes a shaping term on the oldest queue wait, training uses a safe-exploration guardrail that forbids random actions while any phase is starving, and MAX_GREEN of 60 seconds caps any green phase absolutely. Starvation was the failure mode of the first training run; the current architecture was redesigned specifically around it.

Q: What is the sim-to-real gap, honestly?

A: The honest answer has four parts. One: vehicle motion is kinematic, not a full microsimulator; car-following and lane-changing are simplified. Two: the camera is simulated counts with Gaussian noise (sigma 0.7), not real occlusions, rain, or night glare. Three: arrivals are calibrated from real measured headways but the scenario demand shapes are designed, not recorded Lahore traffic. Four: the brain has never controlled a real signal. The gap is real and the team does not hide it; the pilot intersection exists precisely to close it, starting with shadow mode where the brain decides but a human-approved controller executes.

Q: What role does NGSIM data actually play? Be precise.

A: NGSIM is 1,039,689 clean vehicle records from 3,209 real vehicles on US-101, a US highway. The simulator does not replay raw trajectories and the DQN did not train on raw trajectories. What the sim uses is the measured per-lane headway distribution (median and p90 gaps between consecutive arrivals): it samples arrival gaps from that real distribution, rescaled so the mean rate matches each scenario's demand. So NGSIM makes the traffic's burstiness realistic instead of synthetic Poisson; it is calibration data, not training data, and it is American highway data, not Lahore data. Anyone who says otherwise is overselling.

Q: What about the SinD intersection data?

A: Honestly: extracted but not yet integrated. The team processed one track file (611 unique first-seen tracks, median first-appearance gap 1.4 seconds) from the Chinese SinD signalized-intersection dataset, but it is not in the training loop or the benchmark. It is a stated next step for real intersection calibration, not a current claim.

Q: What happens if a camera fails or is blinded?

A: The design answer: the state includes detection noise already, so the brain is trained to be robust to imperfect counts. The deployment answer: a fail-safe hierarchy. If counts go stale or disagree with physics, the intersection falls back to the actuated controller, and if that is unavailable, to fixed time. A failed camera must never mean a dark or frozen intersection; the system degrades to dumber control, never to no control. The watchdog and fail-safe module exists as a prototype; full integration is scheduled before any pilot.

Q: Does it keep learning after deployment?

A: That is the design: the same reward loop runs in shadow mode first, then live, so the policy keeps refining day by day as traffic patterns shift. The continual-learning prototype exists. The honest caveat: online learning on a live intersection needs guardrails (learning rate caps, rollback to the last validated checkpoint) that are designed but not yet battle-tested. The championship demo shows the trained brain; continual learning is the roadmap, not the demo.

Q: Why is the state only 18 numbers? Isn't that too simple?

A: It is simple on purpose. Every feature is something a real camera at a real intersection can actually provide: counts, phase, timing, location, waiting time. A 200-feature state would be academically impressive and undeployable. The 18 features were chosen by failure analysis: the 16-feature version collapsed because it could not

see starvation, so 2 starvation features were added and the collapse disappeared. Features earn their place by fixing real failures.

Q: How do you know the benchmark isn't overfitted?

A: Three protections: the benchmark uses fixed seeds (11, 22, 33) and scenarios the sampler also generates, but evaluation is greedy (no exploration) and the checkpoint only updates on genuine benchmark wins; training scenarios are heavily randomized across locations, accidents, and demand shapes so no single pattern dominates; and the balanced scenario, the hardest one, is still only tied, which is the opposite of what an overfitted benchmark would show.

Q: What hardware does it need at the intersection?

A: A camera per approach (the system is designed for standard traffic cameras), and one small computer in the signal cabinet: the brain is 5,506 parameters doing one 0.02-millisecond inference per second, so even a low-end CPU is overkill. No GPU, no cloud round-trip, no fiber. If the network link dies, the intersection keeps running locally.

Q: Can it handle mixed traffic, rickshaws, motorcycles weaving?

A: Partially, and this is stated as a limitation. The simulator models vehicles as identical kinematic units; it does not model motorcycles filtering between lanes or rickshaws with different acceleration. The camera counting (YOLOv8) detects vehicle classes in real footage, so the sensing side sees the mix; the control side treats them uniformly for now. Modeling heterogeneous traffic properly is explicit future work, and it is one reason a Lahore pilot matters: that is where the real mix lives.

Q: What is the single hardest technical problem you have not solved?

A: The balanced tie: 11.9 vs 11.9 seconds. When all four approaches are equally saturated, the learned policy matches but does not beat good actuated logic. Training continues specifically on this. A judge should hear this voluntarily, because it proves the team knows exactly where the frontier is.

SECTION 4: NON-TECHNICAL Q&A (20 QUESTIONS)

Q: How much does it cost to deploy at one intersection?

A: The honest estimate: cameras on existing poles plus one small cabinet computer plus installation. The team does not quote a hard number because they have not done a real procurement; any number given now would be invented. The structural point is

what matters: no in-road sensors, no road digging, no GPU servers, which removes the most expensive line items of systems like SCOOT and SCATS. A pilot budget is built around commodity hardware, not specialized infrastructure.

Q: Who pays for it?

A: The customer is the city traffic authority or the municipal government, the same bodies that already buy and maintain signals. The model is per-intersection hardware plus a software license or a managed-service fee per corridor per year. Private pilots (a housing society, a campus, a commercial district) are a possible first customer because their procurement is faster than a city's.

Q: What is the business model?

A: Land one pilot intersection, prove measured delay reduction against the existing controller, then sell corridor by corridor. Software margin is high because the brain is tiny and runs on cheap hardware; the defensible asset is the trained policy library per intersection type plus the deployment and calibration process. Long term, a city-wide license with a performance contract: the city pays against measured congestion reduction.

Q: Who are the competitors and how do you differ?

A: SCOOT (Siemens) and SCATS (Transport for NSW, used in many countries) dominate adaptive signals; InSync (Rhythm Engineering) does camera-based adaptive control in the US. All are expensive, built for orderly lane-disciplined traffic, and need costly sensing. SABZ differs on four axes: camera-only with no road digging, a 5,506-parameter brain that runs on commodity CPUs, a hard dilemma-zone safety guarantee, and location-aware state (schools, mosques, Friday patterns) designed for Pakistani traffic first. The team does not claim to out-engineer Siemens today; the claim is a 10x cheaper system designed for the traffic Siemens was not built for.

Q: Why Pakistan, why Lahore, specifically?

A: Because the problem is worst where the systems are weakest. Lahore went from 378,000 to 4.1 million registered vehicles in a decade while its signals still run on fixed timers. Western adaptive systems were tuned for disciplined lanes and homogeneous cars; Lahore has motorcycles, rickshaws, and demand spikes around schools and Friday prayers that those systems never modeled. And there is a principle the team believes: the people who sit in the traffic should be the people who build the fix.

Q: What is the timeline to a real deployment?

A: Stated in phases, not dates pulled from air: phase 1, shadow mode at one intersection (brain decides, existing controller executes, decisions logged and compared); phase 2, supervised live control with instant fallback; phase 3, full autonomous control with monitoring. Each phase needs the previous phase's data. The team will not name a month until the pilot partner is signed, because a fake date is worse than no date.

Q: What do you need from judges or investors?

A: Three things: a pilot intersection (access to one signalized crossing and permission to mount cameras), a traffic-engineering partner to validate the safety case, and compute or funding to keep training and to build the pilot hardware. The ask is specific because the next milestone is specific.

Q: What is the biggest limitation of the system right now?

A: Said plainly: it has never controlled a real signal. Everything is simulation plus offline camera tests. The balanced scenario is tied, not won. Mixed traffic (motorcycles, rickshaws) is not modeled in the controller. The team leads with this because a team that volunteers its limits is more credible than a team that hides them.

Q: What happens in a power outage?

A: The same thing that happens to every signalized intersection today: signals go dark and police manage the crossing. SABZ adds no new failure mode here; the cabinet computer is on the same power as the signal controller. A small UPS that keeps the brain alive for graceful fallback to fixed time is part of the pilot hardware spec.

Q: How do you handle privacy with cameras watching intersections?

A: The system needs counts, not identities: how many vehicles per approach, not who they are or where they go. The design processes video at the edge (in the cabinet), keeps no footage longer than needed for counting, and stores only aggregated counts. No license-plate recognition is part of SABZ, and the team commits to not adding it: the product is traffic flow, not surveillance.

Q: How do you maintain it? Who fixes it when it breaks?

A: The brain is software; updates are pushed like any software update with rollback to the last validated checkpoint. Cameras are cleaned and checked on a schedule like any traffic hardware. The fallback hierarchy (DQN to actuated to fixed) means a software fault degrades control gracefully instead of killing the intersection.

Maintenance is designed to be doable by the city's existing signal maintenance staff, not by AI researchers.

Q: Will it work at a 3-way intersection or a roundabout?

A: The current brain is trained for a 4-way signalized intersection; that is the stated scope. The architecture generalizes: the state is per-approach counts, so a 3-way version is a retraining task, not a redesign. Roundabouts are a different control problem and out of scope for now. The team says what the system is, not what it might one day be.

Q: How is this different from the green-wave corridors some cities already have?

A: Green waves coordinate fixed offsets along a corridor; they are still timers, just synchronized timers. They cannot respond to a queue spilling back at one intersection or an empty road at 2 AM. SABZ decides per intersection per second from live demand. The two ideas compose: adaptive intersections coordinated along a corridor is the endgame, but adaptivity comes first.

Q: What about pedestrians?

A: Honest scope statement: the current system optimizes vehicle delay; pedestrian phases are not in the state or the benchmark. Any real deployment must include pedestrian minimums as hard constraints, the same way the dilemma-zone rule is a hard constraint today. The team does not pretend this is solved; it is on the roadmap before any pilot, because a system that speeds cars by stranding pedestrians is a failure.

Q: How do you measure success in the real world?

A: Average delay per vehicle at the pilot intersection, measured against the same intersection's before-period with its existing controller, same hours, same days of week. Secondary: throughput, stops per vehicle, idle fuel and CO2. The metric discipline from the simulator (fixed seeds, identical conditions) carries over: no cherry-picked hours.

Q: Can the city override or supervise it?

A: Yes, and it must be able to. The design includes a traffic-operations dashboard with live state, the ability to pin a phase (for VIP movement or emergencies), and instant fallback to manual or fixed control. An AI the city cannot supervise is an AI the city will not buy.

Q: What stops someone from copying this?

A: Nothing stops copying the idea; ideas are cheap. The moat is execution: the trained policy library across intersection types, the calibration process that adapts the brain to a new junction in days, the safety validation record, and the pilot data no competitor has from Pakistani intersections. Being first to real deployment data in this market is the advantage.

Q: Is the system fair to all roads, or does it favor the big road?

A: Fairness is engineered, not hoped for: the starvation features let the brain see waiting time, the reward punishes the oldest wait, MAX_GREEN caps any phase at 60 seconds, and the safety rule protects every approach equally. In the asymmetric benchmark the brain serves the quiet cross road instead of starving it; that is measured, not claimed.

Q: What is your plan if the pilot shows no improvement?

A: Then the pilot did its job: it told the truth. The team would publish the negative result, diagnose whether the gap is sensing, modeling, or the policy, and fix the actual cause. A team that has a plan for being wrong is a team a city can trust with its intersections.

Q: Where is SABZ in five years, if everything works?

A: Every signalized intersection in Lahore running an adaptive brain, then other Pakistani cities, then export to the region's cities with the same traffic character. The bigger vision: intersections that learn their city's rhythm the way a good traffic policeman does, but at every crossing at once, never tired, never bribed, never absent.

SECTION 5: TRAP QUESTIONS (10 HOSTILE QUESTIONS)

How to handle these: stay calm, concede what is true, answer the real question, never get defensive.

Q: Isn't this just a timer with extra steps?

A: A timer does the same thing at 3 AM and 6 PM and cannot see a single car. SABZ counts real vehicles every second and its decisions change with what it sees: it holds green for a platoon it watches approaching and cuts green on an empty road. The benchmark measures the difference: on school-zone traffic the brain cuts average

delay from 5.3 to 5.0 seconds against the best hand-tuned actuated logic. Extra steps that measurably cut waiting are not decoration.

Q: Your data is American highways. How is this Pakistan?

A: Correct, and the team says so before being asked. NGSIM is US-101 headway data used only to make arrival burstiness realistic in the simulator; it is calibration, not training, and nobody claims it is Lahore traffic. What is Pakistani is the problem framing: the location features for schools, mosques, and Friday surges, the design for mixed chaotic traffic, and the camera-only deployment with no road digging. The Lahore pilot is precisely how real Pakistani data enters the system.

Q: Four students built this in weeks. Why hasn't Siemens done it?

A: Siemens built for its market: orderly Western traffic, expensive sensors, high-margin hardware. Nobody paid them to solve a Lahore intersection with a 5,506-parameter model on a cheap CPU. Students can move where incumbents have no incentive to go. The team does not claim to out-engineer Siemens; they claim an incumbent blind spot, and blind spots are where newcomers win.

Q: Your balanced scenario is a tie. So the AI adds nothing half the time?

A: The tie is on one of three scenarios, and it is disclosed voluntarily. On asymmetric and school traffic the brain wins; on balanced it matches the best hand-tuned controller. Matching the best existing system at its strongest point while beating it elsewhere is not 'nothing'. And the tie is exactly why training continues: the team knows the frontier and is working it.

Q: What if the AI makes a mistake and causes a crash?

A: This is why the safety rule is not learned. The dilemma-zone veto sits outside the neural network: no weight update, no bug in the policy, no bad day can switch a green under a car within 70 metres of the stop line. The AI proposes; the physics rule disposes. A crash from an unsafe switch is prevented by construction, not by the AI being smart enough.

Q: Couldn't a good traffic engineer just tune the timers better?

A: A tuned timer is still a timer: it is right on average and wrong every specific second. The actuated controller in our benchmark is the good-engineer answer, hand-tuned if-then logic, and the brain beats or matches it. Tuning finds the best fixed answer; learning finds a different answer for every second.

Q: This needs cameras everywhere. What about cost and privacy?

A: Cost: cameras on existing poles, one small computer per cabinet, no road digging, no in-ground sensors; that removes the most expensive parts of incumbent systems. Privacy: the system counts vehicles, it does not identify them; video is processed at the edge and only aggregate counts are kept. No plate recognition, by design commitment, not by accident.

Q: Simulation results don't count. Where is the real-world proof?

A: Agreed: simulation is evidence, not proof. That is why the next milestone is a single pilot intersection in shadow mode, measuring real delay against the existing controller. The team does not present simulation as deployment; they present it as the reason a pilot is worth running. Ask what the pilot measures, and the answer is ready: average delay per vehicle, same intersection, same hours, before and after.

Q: What stops the AI from learning something dangerous in live operation?

A: Two things. First, the dangerous action is not available to learn: the safety veto is outside the network, so 'switch under a car' never executes no matter what the policy outputs. Second, live learning runs with guardrails: capped learning rates and instant rollback to the last validated checkpoint. The brain can get dumber in deployment; it cannot get dangerous.

Q: Why should we believe students can deploy this in a real city?

A: Believe the artifacts, not the ages. The DQN is written from scratch, the safety proof is in the code, the benchmark is reproducible, the limits are volunteered. Deployment is not claimed; a phased pilot with shadow mode, supervision, and fallback is proposed. Judge the engineering, and judge whether the team tells the truth about what is done and what is not. That is the actual test.

END OF GUIDE. Good luck. Tell the truth, show the numbers, and let the demo do the talking.